

Beijing-Dublin International College



COMP3038J - Multi-Paradigm Programming
Individual Assignment 2026: League Management System

Assignment Details

Due date: 11th of December 2026 at 17:00
Language: Solution must be completed in Scala (version 3)

Task Description

Your task is to design and implement a system for running the Riverside Amateur Football League, a league of ten teams that each play every other team twice during the season (once at home and once away). The league administrator is responsible for generating the fixture list for each season and assigning a referee to every fixture, referees record the results of the matches they officiate, and each team has a manager who looks after the team's squad of players. Everybody involved wants to see the league table, the results and who is scoring the goals.

The important thing to understand about this system is that **most of what users want to see is calculated rather than stored**. The league table, the form of each team and the top scorers are never saved in any file. They must all be calculated from the results and match events every time they are needed. This makes the assignment a good opportunity to write a core of pure functions that turn lists of data into useful answers, keeping the input and output of the program at the edges.

The system you are tasked with building should be interacted with using the command line, although if you want to **independently** learn another technology such as GUI or web interfaces an amount of bonus marks will be available. The system should have the following functionality:

- The ability for users to login using their user ID and a password
- The ability to view the current league table (all users)
- The ability to view the fixtures and results, either for a round or for a single team (all users)
- The ability to view the top goal scorers in the league (all users)
- The ability to view their squad, including the goals of every player, and the form of the team (team managers only)
- The ability to register a new player in their squad (team managers only)
- The ability to record the result of a fixture they were assigned to, including the goals (referees only)
- The ability to start a new season by generating a new list of fixtures, with a referee assigned to every fixture (administrator only)

League Rules

Points and the League Table

A team is awarded 3 points for a win, 1 point for a draw and 0 points for a loss. The league table shows, for each team, the number of matches played (P), won (W), drawn (D) and lost (L), goals for (GF), goals against (GA), goal difference (GD = GF − GA) and points (Pts). Teams are ordered using the following rules, where each rule is used only if the teams are still level after all of the previous rules:

1. Most points
2. Greatest goal difference
3. Most goals scored
4. Most points earned in the matches played between the teams that are still level (head-to-head)
5. Alphabetical order of the team name

The provided data includes teams that can only be separated by goals scored, and the top two teams can only be separated by their head-to-head result, so make sure that you implement all of these rules.

Form

The form of a team is the result (W, D or L) of each of its last five played matches, oldest first, so that the most recent result is on the right (e.g. `WDDLW`).

Match Events and Goals

Every goal in a match is recorded as a match event. A goal is either a `Goal`, a `PenaltyGoal` or an `OwnGoal`. An own goal is recorded against the player who scored it (and their team) but counts as a goal for the **opposing** team. The score of a match must always agree with its goal events. Own goals do not count towards a player's total in the list of top scorers.

Generating Fixtures

When the administrator starts a new season, the system must generate a complete new list of fixtures for the ten teams. The new list must meet the following requirements:

- The season has 18 rounds and every team plays exactly once in each round
- Every pair of teams meet twice, once at the home ground of each
- Rounds 10 to 18 are the same as rounds 1 to 9, but with home and away swapped
- No team plays more than two home matches, or two away matches, in a row within rounds 1–9 (or within rounds 10–18)
- Round 1 is played on a Saturday chosen by the administrator, with each following round on the next Saturday, all matches kicking off at 15:00
- Every fixture is assigned a referee, and no referee is assigned to more than one fixture in the same round
- The fixtures are shared out between the referees as evenly as possible (with the six referees in the provided data, each referee is assigned exactly 15 fixtures)

Before generating the new fixtures, the existing fixture, result and event files should be copied into a folder named after the season that has finished (e.g. `archive/2026-27/`).

Recording Results

A referee may only record the result of a fixture that they have been assigned to, that has a status of `Scheduled`, and whose kick-off time has passed. The referee enters the score and then each event (minute, player and type of event). Before it is saved the system must check that every player belongs to one of the two teams, that minutes are between 1 and 90, and that the goal events agree with the score. Once saved the fixture has a status of `Played`.

Registering Players

A squad can contain at most 25 players, and no two players in a squad can have the same shirt number (1–99). New player IDs follow on from the highest existing ID (e.g. `P0161`), and the date of birth is entered in the format `yyyy-MM-dd`.

Persistent Data Storage

When the program is first executed, the records of the system should be loaded from a number of files that are saved in csv format. When the user exits the program, the records from the system should be written back into these same files with any new data added to the files. This will allow the program to remember data between executions. None of the calculated values described above (table, form, scorers) should be written to any file.

Provided Data Files

I have prepared the following data files to represent the data of the system. The data represents the league after the matches of the 31st of October 2026 (round 9). None of the files contain a header line.

- `staff.csv` - Contains the user accounts of the system. This includes the following:
  - The user ID (`A` for the administrator, `R` for referees and `M` for team managers)
  - The name
  - The role of the user (`Administrator`, `Referee` or `TeamManager`)
  - The team ID of the team that they manage (empty unless the user is a team manager)
  - The encoded password of the user for logging into the system[1]
- `teams.csv` - Contains the team ID, the name, the name of the home ground and the year the team was founded.
- `players.csv` - Contains the player ID, the team ID, the name, the date of birth, the position (`Goalkeeper`, `Defender`, `Midfielder` or `Forward`) and the shirt number.
- `fixtures.csv` - Contains the fixtures of the season. This includes the following:
  - The fixture ID
  - The round number (1–18)
  - The kick-off date and time
  - The team ID of the home team, followed by the team ID of the away team
  - The user ID of the referee assigned to the fixture
  - The status of the fixture (`Scheduled` or `Played`)
- `results.csv` - Contains the fixture ID, the goals scored by the home team, the goals scored by the away team, the user ID of the referee who recorded the result and the date and time it was recorded.
- `events.csv` - Contains the events of the played matches. Each line contains the fixture ID, the minute of the event, the team ID of the player, the player ID and the type of event (`Goal`, `PenaltyGoal`, `OwnGoal`).

All dates in the files use the ISO 8601 standard format: dates are written as `yyyy-MM-dd` (e.g. `2026-09-05`) and dates with a time as `yyyy-MM-ddTHH:mm` (e.g. `2026-09-05T15:00`). These can be converted directly using the `LocalDate.parse` and `LocalDateTime.parse` methods of the `java.time` package, and are written back in the same format by their `toString` methods. Any dates entered by the user should also be in the format `yyyy-MM-dd`.

Passwords

The user accounts have their passwords recorded in a form that is safe for storage. That is, only an encoded version of the password is remembered by the system. The library that I am using for this is named `scalapass` and can be included in your project by adding the following line to the bottom of the `build.sbt` file in your project.

[1] More on this later

```
libraryDependencies += "com.outr" %% "scalapass" % "1.2.8"
```

Once this has been added, you can import the factory class with the following import statement:

```
import com.outr.scalapass.Argon2PasswordFactory
```

This then allows you to create a factory object to interact with the library:

```
val factory = Argon2PasswordFactory()
```

To convert a plain text password into an encoded hash, you can use the `hash` method of the factory, passing it a string containing the password as a parameter:

```
factory.hash("password")
```

Once the password has been encoded, there is no way to reverse this process. However, we can check if a plain text password we supply encodes to the same encoded hash using the `verify` method:

```
factory.verify(textPassword, encodedPassword)
```

This code will return the Boolean value true if the text password that we passed encodes into the same hash as is remembered in the encoded password string. It should be noted that some care is required when loading passwords from the file as the encoded passwords will contain comma characters that may cause you to not load the full encoded string correctly. The actual user IDs and passwords are listed in the file `users.csv`.

User Input

All user input should be robust. This means that when a user is requested to enter a value of some kind the text they enter must be verified and if not appropriate the user should be repeatedly prompted until they have successfully entered the data. This includes user IDs, menu selections, scores, minutes, player IDs, shirt numbers and dates. For passwords, the user should only be allowed three incorrect attempts after which the program should exit.

Design Requirements

In addition to the functional requirements described above, your design will also be assessed on the following:

- The league table, form, top scorers, and fixture generation must each be calculated by **pure functions** that take the loaded data as parameters. These functions must not read files, print to the screen or use mutable state.
- Functions that can fail (e.g. parsing a line of a file, finding a team, validating a result) should return an `Option`, `Try` or `Either` rather than throwing exceptions or returning `null`.
- Calculations over collections should be written using higher order functions (`map`, `filter`, `foldLeft`, `groupBy`, `sortWith`, ...) rather than loops that update variables.

Example

This assignment is not graded by the test system and as such there is no set output that you must conform to. This section is an example of how the execution of the program **may** appear when implemented.

```
Welcome to the Riverside Amateur Football League
Please enter your user ID: M007
Please enter your password: pasword
Incorrect password, 2 attempts remaining
Please enter your password: password

User:  M007 - Mei Wilson
Role:  Team Manager (Glenarra Town)
Please choose one of the following options:
    1. View the league table
    2. View fixtures and results
```

```
    3. View top scorers
    4. View my squad
    5. Register a new player
    6. Exit the program
Enter choice: 1
Pos  Team                 P   W   D   L   GF  GA   GD  Pts
  1  Inchbeg Harriers     9   6   2   1   20  11   +9   20
  2  Glenarra Town        9   6   2   1   20  11   +9   20
  3  Harbourside FC       9   4   3   2   16  19   -3   15
  4  Kilbrook City        9   4   2   3   22  19   +3   14
  5  Carrigvale Celtic    9   3   3   3   15  15   +0   12
  6  Elmwood Wanderers    9   2   6   1   12  12   +0   12
  7  Fernhill Albion      9   2   4   3    8   7   +1   10
  8  Ashford Rovers       9   2   2   5    9  11   -2    8
  9  Dunmore United       9   1   4   4   13  18   -5    7
 10  Bellmount Athletic   9   0   2   7    9  21  -12    2
```

```
Enter choice: 4
Glenarra Town   Form: DWWWD   Next: Round 10 vs Dunmore United (H) on 2026-11-07
No.  Name                 Position     Goals
  1  Una Thompson         Goalkeeper       0
  4  Tomasz Hughes        Forward          1
  5  Oisin Novak          Defender         1
  6  Una Kennedy          Midfielder       2
  8  Mei Rossi            Midfielder       1
 12  Zoe Maguire          Midfielder       2
 16  Conor Callaghan      Defender         0
 ...
```

Assessment

The final marking scheme may vary slightly but will be relatively similar to the one below. Your submission should contain the following files

- A zip file containing the entire project code and any other files required for the code to be executed (exported from IntelliJ IDEA)
- A report explaining what you have achieved for each of the criteria in the marking scheme (estimate fail, pass or excellent) based on a template which will be provided

Each file should be named only with your UCD student number and the correct file extension (e.g. 1234567.zip, and 1234567.pdf).

Zip File

This must contain all of the code required to execute the project. If I am unable to execute your code, it will reduce your grade. If there are any requirements to setup and execute your code you should include these in the code of the project somewhere easy to find. A `README.md` file would be the ideal place to put instructions like this.

I will be testing the code using IntelliJ IDEA (unless you specifically mention that I should use another IDE). I would recommend that you create the project using the IntelliJ IDEA IDE and save the entire project folder to a zip file for submission.

Assignment Report

This document should be in PDF format and contain your name and student number as well as a table explaining what you estimate of how your project will be graded against the marking criteria in the next section. For each of the criteria mark an X under Fail, Pass, or Excellent and optionally include some text to explain why you believe you have merited that level. You may also include details in this report of any additional functionality that you have added that you believe should be worth extra credit.

Marking Scheme

The marking scheme shown in table 1 is subject to change. This means that it may be changed at any time without notice if I feel some parts were too easy or too hard and additional criteria may be added. The final grade from this section will be based on a weighted sum of the individual parts. The weights for each part will be based on the difficulty and importance.

The text explaining each can be used as a guide to the amount of work expected for the different parts of the assignment.

Additional Criteria

In addition to the above, your submission will also be considered under some more criteria. These however are only used to reduce your grade if you have performed poorly and will otherwise have no effect.

- Naming of variables, methods, class, interfaces, enumerated types or anything else in your code
- Following naming conventions for the above
- Code formatting and commenting[2]
- Correct formatting of submitted files (e.g you will lose points for submitting a doc file instead of a pdf or rar instead of zip)
- Incorrect naming of any submitted files
- Not including name and student number in report (or including an incorrect student number)

[2] I do not expect a large amount, but at the least there should be an explanation of the use of each class

Item	Fail	Pass	Excellent
		Design and Cohesion	
Package Structure	All of the code is within a single package		Code for modelling, calculations, file IO, and user interaction are stored in separate packages
Modelling - Attributes	Many/Most values are represented as strings, when they should be enumerated types	Some values are represented as strings, when they should be enumerated types	All values that could be represented as enumerated types are
Modelling - Classes	Many/Most classes are represented using mutable state	Some classes are represented using mutable state	All classes are represented using immutable state where appropriate
Modelling - Functional Objects	The functional objects approach was not used to design classes	Some objects were designed using the functional objects approach	All appropriate objects were designed using the functional objects approach
Pure Calculations	Calculations are mixed with file IO or printing, or rely on mutable state	Most calculations are pure functions, but some read files, print or use mutable state	The table, form, scorers and fixture generation are all pure functions of the loaded data
Error Handling	Exceptions or `null` are used to signal failure, or the program crashes on bad input	`Option`, `Try` or `Either` are used in some places	Failure is always represented with `Option`, `Try` or `Either`, and is handled with pattern matching or for comprehensions
Higher Order Functions	Calculations are mostly written with loops updating variables	Some calculations use higher order functions	Calculations over collections use higher order functions and no loops update variables
Methods	Methods are poorly named, do not use **camelCase** and are frequently longer than 30 lines of code	Some methods have descriptive names, and are mostly not larger than 30 lines of code	All methods have descriptive names, are written in **camelCase** and are mostly not larger than 30 lines of code
		Other Criteria	
Input	User input is not completed	User input is completed but it was not robust	User input is completed and user failures resulted in additional attempts to enter the information
Menu	No menu or other options are shown	Menu is shown, but may not contain all options or contains options that are not appropriate for the user that logged in	Menu is shown and it contains all options that are appropriate for the user that logged in
Data	Data is not loaded or saved by the program	Data is loaded from the files, but is not saved after actions or when the program closes	Data is loaded when the program starts and is saved either after a change or when the program closes
Abilities - Login	Login is not implemented		Login is implemented and working correctly
Abilities - League Table	The league table cannot be viewed	The table is shown but some of the ordering rules are not applied	The table is shown with all of the ordering rules (including head-to-head) applied
Abilities - Fixtures, Results and Scorers	Fixtures and results cannot be viewed	Fixtures and results can be viewed, but not by both round and team, or the top scorers are missing	Fixtures and results can be viewed by round and by team, and the top scorers are listed (excluding own goals)
Abilities - Squad and Form - Manager	Managers cannot view their squad	The squad is shown, but goals or form are missing or incorrect	The squad is shown with correct goals, and team form
Abilities - Register Player - Manager	Managers cannot register players	Players can be registered, but squad size or shirt number rules are not checked	Players can be registered and all rules are checked
Abilities - Record Result - Referee	Referees cannot record results	The score can be recorded, but events are not recorded or not validated	The score and events are recorded, and all of the checks (assigned referee, kick-off passed, team membership, score agreement) are applied
Abilities - New Season - Administrator	A new fixture list cannot be generated	A fixture list is generated where every pair of teams meet home and away, but some of the other requirements (including the assignment of referees) are not met	A fixture list is generated that meets all of the requirements, including the assignment of referees, and the previous season is archived
Appraisal	No Appraisal was included	The appraisal was very inaccurate, and claimed the project was better than the code showed	The appraisal was very accurate, and highlighted areas where the project met or exceeded the project specifications as well as areas where it did not meet them (if applicable)

Table 1: Indicative Marking Scheme for Assignment